

Lab 05 – File I/O, Interfaces, Factory Pattern

Title: Data Persistence using Text Files and Factory Pattern

Objectives

The main goal here was to implement persistent storage so that employee data survives when the application is closed and reopened. I also learned about the Factory Design Pattern and how to use configuration files (app.config) to make the application flexible. This was a significant step from a simple in-memory application to one that can actually store and retrieve data.

Implementation

File Storage Implementation

```
using SalariiModel;

using System;

using System.Collections.Generic;

using System.IO;

using System.Linq;

namespace SalariiStocare
{
    public class AdministrareAngajatiFisierText : IstocareData
    {
        private readonly string _numeFisier;

        private int _nextId = 1;
```

```

public AdministrareAngajatiFisierText(string numeFisier)
{
    _numeFisier = numeFisier;
    if (!File.Exists(_numeFisier))
        File.Create(_numeFisier).Close();
    else
        _nextId = GetNextId();
}

private int GetNextId()
{
    var angajati = GetAngajati();
    return angajati.Any() ? angajati.Max(a => a.Id) + 1 : 1;
}

public void AddAngajat(Angajat angajat)
{
    angajat.Id = _nextId++;
    using (StreamWriter sw = new StreamWriter(_numeFisier, true))
    {
        sw.WriteLine(angajat.ConversieLaSirPentruFisier());
    }
}

public List<Angajat> GetAngajati()
{
    var angajati = new List<Angajat>();
    if (!File.Exists(_numeFisier)) return angajati;

    using (StreamReader sr = new StreamReader(_numeFisier))

```

```

{
    string line;
    while ((line = sr.ReadLine()) != null)
    {
        if (!string.IsNullOrEmpty(line))
            angajati.Add(new Angajat(line));
    }
}
return angajati;
}

```

```

public List<Angajat> CautaDupaNume(string nume)
{
    return GetAngajati().Where(a =>
        a.Nume.Contains(nume, StringComparison.OrdinalIgnoreCase) ||
        a.Prenume.Contains(nume, StringComparison.OrdinalIgnoreCase)
    ).ToList();
}

```

```

public List<Angajat> CautaDupaFunctie(string functie)
{
    return GetAngajati().Where(a =>
        a.Functie.Contains(functie, StringComparison.OrdinalIgnoreCase)
    ).ToList();
}

```

```

public bool ModificaAngajat(int id, Angajat angajatNou)
{
    var angajati = GetAngajati();
    var existing = angajati.FirstOrDefault(a => a.Id == id);
}

```

```
if (existing == null) return false;
```

```
existing.Nume = angajatNou.Nume;
```

```
existing.Prenume = angajatNou.Prenume;
```

```
existing.Functie = angajatNou.Functie;
```

```
existing.Departament = angajatNou.Departament;
```

```
existing.TipSalariu = angajatNou.TipSalariu;
```

```
existing.SalariuBrut = angajatNou.SalariuBrut;
```

```
existing.TarifOra = angajatNou.TarifOra;
```

```
existing.OreLucrate = angajatNou.OreLucrate;
```

```
existing.Bonusuri = angajatNou.Bonusuri;
```

```
existing.DataAngajarii = angajatNou.DataAngajarii;
```

```
// Rewrite the entire file
```

```
using (StreamWriter sw = new StreamWriter(_numeFisier, false))
```

```
{
```

```
    foreach (var a in angajati)
```

```
        sw.WriteLine(a.ConversieLaSirPentruFisier());
```

```
}
```

```
return true;
```

```
}
```

```
public bool StergeAngajat(int id)
```

```
{
```

```
    var angajati = GetAngajati();
```

```
    var existing = angajati.FirstOrDefault(a => a.Id == id);
```

```
    if (existing == null) return false;
```

```
    angajati.Remove(existing);
```

```
    using (StreamWriter sw = new StreamWriter(_numeFisier, false))
```

```

    {
        foreach (var a in angajati)
            sw.WriteLine(a.ConversieLaSirPentruFisier());
    }
    return true;
}
}
}

```

Serialization Methods in Angajat Class

```

public class Angajat
{
    // ... properties ...

    private const char SEPARATOR = '|';

    // Serialization - convert object to file line
    public string ConversieLaSirPentruFisier()
    {
        return string.Join(SEPARATOR,
            Id, Nume, Prenume, Functie, Departament, TipSalariu,
            SalariuBrut, TarifOra, OreLucrate, Bonusuri, DataAngajarii.ToString("yyyy-MM-dd"));
    }

    // Deserialization - create object from file line
    public Angajat(string linieFisier)
    {
        var parts = linieFisier.Split(SEPARATOR);
        Id = int.Parse(parts[0]);
        Nume = parts[1];
    }
}

```

```

Prenume = parts[2];

Functie = parts[3];

Departament = Enum.Parse<Departament>(parts[4]);

TipSalariu = Enum.Parse<TipSalariu>(parts[5]);

SalariuBrut = double.Parse(parts[6]);

TarifOra = double.Parse(parts[7]);

OreLucreate = int.Parse(parts[8]);

Bonusuri = double.Parse(parts[9]);

DataAngajarii = DateTime.Parse(parts[10]);
}
}

```

App.config (in UI project)

```

<?xml version="1.0" encoding="utf-8" ?>

<configuration>

  <appSettings>

    <add key="FormatSalvare" value="txt"/>

    <add key="NumeFisier" value="Angajati"/>

  </appSettings>

</configuration>

```

Why I Used This Approach

Interface Implementation: Both `AdministrareAngajatiMemorie` and `AdministrareAngajatiFisierText` implement the same `IStocareData` interface. This means the UI code doesn't care which storage mechanism is used – it just calls the interface methods.

Pipe-Delimited File Format: I used the pipe character (`|`) as a delimiter. This is simpler than CSV and avoids issues with commas in text fields. The format is human-readable, which is helpful for debugging.

Serialization/Deserialization Methods: By adding `ConversieLaSirPentruFisier` and a constructor that takes a string, I encapsulate the conversion logic within the `Angajat` class. This keeps the storage class focused on file operations.

Factory Pattern: The `StocareFactory` class abstracts the creation logic. The rest of the application doesn't care about the storage implementation details. This makes it easy to switch storage mechanisms by changing a configuration value.

Configuration File: Using `app.config` makes the application configurable without recompiling. I can switch between memory and file storage just by changing a setting.

File Handling: The constructor checks if the file exists and creates it if it doesn't. It also calculates the next available ID by reading all employees – this ensures IDs remain unique across sessions.

Testing & Results

I tested the file storage by:

1. Adding employees through the application
2. Closing the application
3. Reopening it – all employees were correctly loaded from the text file

The file format was human-readable, which helped with debugging:

```
1|Popescu|Ion|Programator|IT|Lunar|8000|0|0|0|2024-01-15
```

```
2|Ionescu|Maria|Manager|Marketing|Orar|50|160|100|200|2024-02-01
```

Switching between memory and file storage via `app.config` worked seamlessly – the rest of the code didn't need any changes.

Reflection

The Factory Pattern is very powerful for this kind of situation. The rest of the application only depends on the `IStocareData` interface, not on any concrete implementation. This means I could switch to a database later without changing any UI code. The `app.config` approach also makes the application configurable without recompiling. The separation of serialization logic into the model class keeps the storage class clean and focused.